



PAUL SABATIER

RAPPORT DE STAGE

Process mining sur des données sportives

Élèves :

Mael LATOR-ROUBIN
Sergio PILOTTO MANRESA

Tuteur :

Emmanuelle CLAEYS

14 avril 2025

Table des matières

1	Introduction	2
2	Les données	3
2.1	Détails des données	3
2.2	Datasets	4
2.3	Sequençage des données	4
2.4	Représentation des séquences	5
3	Clustering de séquence	7
3.1	Distance	7
3.1.1	Caractéristiques	7
3.1.2	Distance de Levenshtein	8
3.1.3	Distance LCS	8
3.2	Classification par Ascendant Hiérarchique	9
3.2.1	Principe du CAH	10
3.2.2	Algorithme	10
3.2.3	Exemple et dendrogramme	11
3.3	Elbow Method	11
3.4	Représentant d'un cluster	12
3.4.1	Représentant-centroïde	12
3.4.2	Représentant-médoïd	13
4	Analyse	15
4.1	Analyse générale	15
4.1.1	Possession	15
4.1.2	Les actions	16
4.1.3	Gain de distance	17
4.2	Analyse de processus (post-clustering)	18
4.2.1	Creation des événements dans bupaR	18
4.2.2	Matrice de précédence	19
4.2.3	Graphe de processus	20
4.3	Synthèse	22
5	Conclusion	23
6	Bibliographie	24
7	Annexe	25

1 Introduction

Le rugby est un sport collectif complexe et dynamique qui exige une coordination et stratégie précise de la part des joueurs. L'analyse des actions d'un match de rugby peut fournir des informations précieuses sur les performances des joueurs et des équipes, ainsi que sur les stratégies et tactiques employées. Le process mining est une technique d'analyse de données basée sur l'étude de séquences qui peut être utilisée pour extraire des connaissances à partir des données d'événements, telles que les données de suivi des actions dans un match de rugby. Nous verrons alors quel est le rôle du process mining dans l'analyse de données sportives ?

Dans ce rapport, nous présenterons une analyse des actions d'un match de rugby avec du process mining. Dans un premier temps, nous introduirons les données utilisées et comment nous les avons transformées. Ensuite, nous aborderons les différentes étapes rencontrées afin d'appliquer un clustering à nos données. Nous terminerons par une analyse du dataset.

2 Les données

Le dataset est issu d'un match de rugby opposant le Stade Toulousain à Leinster lors de la demi-finale de la coupe d'Europe de rugby (29/04/2023).

2.1 Détails des données

	team_id	ps_timestamp	ps_endstamp	action	actionName	x_coord	y_coord	sequence_id	date_posix_start	date_posix_end	zoneAction
1	2350	0.00	0.00	14	N-Restart	50	34	1	1970-01-01 00:00:00	1970-01-01 00:00:00	
2	2350	5.92	5.92	2	D-Tackle	88	53	1	1970-01-01 00:00:05	1970-01-01 00:00:05	
3	2350	6.88	6.88	2	D-Tackle	88	53	1	1970-01-01 00:00:06	1970-01-01 00:00:06	
4	2300	7.20	7.20	1	O-Carry	7	9	1	1970-01-01 00:00:07	1970-01-01 00:00:07	
5	2300	7.20	15.84	23	D-Ruck	14	15	1	1970-01-01 00:00:07	1970-01-01 00:00:15	
6	2300	7.68	7.68	43	N-Ruck OOA	14	15	1	1970-01-01 00:00:07	1970-01-01 00:00:07	
7	2300	10.68	10.68	43	N-Ruck OOA	14	15	1	1970-01-01 00:00:10	1970-01-01 00:00:10	
8	2300	12.80	12.80	43	N-Ruck OOA	14	15	1	1970-01-01 00:00:12	1970-01-01 00:00:12	
9	2350	13.28	13.28	43	N-Ruck OOA	86	53	1	1970-01-01 00:00:13	1970-01-01 00:00:13	
10	2300	16.80	16.80	4	N-Kick	12	15	1	1970-01-01 00:00:16	1970-01-01 00:00:16	
11	2300	16.80	16.80	45	D-Defensive Exit	12	15	1	1970-01-01 00:00:16	1970-01-01 00:00:16	
12	2300	4.52	21.00	15	NA-Possession	7	9	1	1970-01-01 00:00:04	1970-01-01 00:00:21	
13	2350	21.00	68.12	46	O-Opposition 22 Entry	59	67	2	1970-01-01 00:00:21	1970-01-01 00:01:08	
14	2300	23.68	23.68	2	D-Tackle	42	8	2	1970-01-01 00:00:23	1970-01-01 00:00:23	
15	2350	24.08	24.08	1	O-Carry	59	67	2	1970-01-01 00:00:24	1970-01-01 00:00:24	
16	2350	24.08	24.08	3	O-Pass	57	58	2	1970-01-01 00:00:24	1970-01-01 00:00:24	
17	2300	26.48	26.48	2	D-Tackle	43	17	2	1970-01-01 00:00:26	1970-01-01 00:00:26	
18	2350	27.96	27.96	1	O-Carry	57	58	2	1970-01-01 00:00:27	1970-01-01 00:00:27	
19	2300	27.88	27.88	2	D-Tackle	43	17	2	1970-01-01 00:00:27	1970-01-01 00:00:27	
20	2350	27.96	31.00	23	D-Ruck	58	51	2	1970-01-01 00:00:27	1970-01-01 00:00:31	
21	2350	29.92	29.92	43	N-Ruck OOA	58	51	2	1970-01-01 00:00:29	1970-01-01 00:00:29	
22	2300	30.80	30.80	43	N-Ruck OOA	42	17	2	1970-01-01 00:00:30	1970-01-01 00:00:30	
23	2350	31.44	31.44	3	O-Pass	56	51	2	1970-01-01 00:00:31	1970-01-01 00:00:31	
24	2350	34.04	34.04	1	O-Carry	58	51	2	1970-01-01 00:00:34	1970-01-01 00:00:34	
25	2300	33.84	33.84	2	D-Tackle	43	31	2	1970-01-01 00:00:33	1970-01-01 00:00:33	
26	2300	33.96	33.96	2	D-Tackle	43	31	2	1970-01-01 00:00:33	1970-01-01 00:00:33	

FIGURE 1 – Dataset

Afin de comprendre le contenu du dataset, voici un descriptif des différentes colonnes que nous avons jugés pertinentes pour notre analyse :

- team_id (catégoriel, {2300, 2500}) : identifiant de l'équipe, 2300 pour le Leinster et 2350 pour le Stade Toulousain.
- ps_timestamp (quantitatif, [0, 7000]) : temps en secondes du début de l'action.
- ps_endstamp (quantitatif, [0, 7000]) : temps en secondes de la fin de l'action.
- sequence_id (catégoriel, [0, 110]) : numéro de séquence à laquelle appartient l'action.
- x_coord et y_coord (quantitatif, [-10, 110]*[0, 70]) : coordonnées de l'action, x étant la longueur du terrain et y la largeur.
- action (catégoriel, [0, 50]) : identifiant de l'action.
- actionName (catégoriel, chaîne de caractères) : label de l'action.
- date_posix_start (quantitatif, format AAAA-MM-JJ hh:mm:ss) : date du début de l'action.
- date_posix_end (quantitatif, format AAAA-MM-JJ hh:mm:ss) : date de la fin de l'action.
- zoneAction (catégoriel, {0, 1, 2}) : zone où l'action a eu lieu (0 pour la zone des 22 mètres du Stade Toulousain, 1 pour le milieu et 2 pour la zone des 22 mètres du Leinster).

Nous avons ajouté les colonnes *sequence_id*, *date_posix* et *zoneAction* nous mêmes pour des raisons de convenabilité : *sequence_id* pour savoir à quelle séquence (voir 2.3)

appartient une action, *date_posix* pour utiliser une mesure temporelle acceptée par la librairie BupaR et *zoneAction* pour savoir dans quelle zone du terrain a eu lieu une action.

De plus, nous avons ajouté des indicateurs devant les actions : O pour les actions offensives, D pour les défensives et N pour celles qui sont neutres. Voici la signification des différentes actions :

Action	Signification
Restart	Engagement
Collection	L'équipe récupère la balle
Tackle	Plaquage
Carry	Joueur avance avec la balle
Ruck	Action qui marque le début du ruck
Ruck OOA	Un joueur rejoint le ruck
Kick	Tir au pied
Defensive Exit	Dégagement (si précédé par Kick il s'agit d'un dégagement au pied)
Possession	Changement de possession
Missed Tackle	Plaquage raté
Attacking Qualities	Geste offensif
Defensive Action	Action défensive
Pass	Passe
Opposition 22 Entry	L'équipe entre dans la zone proche de l'embut de l'adversaire
Lineout Throw	Lancement de la touche
Lineout Take	Prise de la touche
Penalty Conceded	Faute pour l'adversaire
Goal Kick	Transformation
Turnover	Turnover
Scrum	Mêlée
Card	Carton
Maul	Mêlée spontanée
Try	Essai
Counter Attack	Contre-attaque

2.2 Datasets

Dans un premier temps, nous avons utilisé un dataset principal, noté *ds_final*, construit à partir d'un nettoyage des données du dataset brute d'origine plus l'ajout de données supplémentaires (voir 2.1). Ceci nous a permis de garder seulement les informations que nous avons considéré pertinentes et éliminer celles qui nous semblaient redondantes.

C'est à partir de ce dataset que nous avons séparé les données en deux autres datasets, *ds_2300* et *ds_2350*. Ils contiennent les séquences de Leinster et du Stade Toulousain respectivement, afin de manipuler les données propres aux équipes plus aisément.

2.3 Séquençage des données

Pour analyser les données sous forme de processus, nous aurons besoin de séquences d'actions et donc transformer les données brutes. Il s'agit de regrouper des actions pour

pouvoir comparer les groupes entre eux. L'objectif de ce séquençage est de pouvoir faire ressortir des liens temporels entre deux actions.

Pour la suite, on considérera la longueur d'une séquence par le nombre d'actions qu'elle englobe.

Pour créer des séquences cohérentes, il faut satisfaire certaines conditions :

- Le nombre de séquences finales créées doit être suffisant pour pouvoir en tirer des conclusions générales.
- Les séquences doivent être de longueur suffisante pour être significative : pas de séquence avec une ou deux actions seulement.
- La création des séquences doit se baser sur un événement majeur.

Il existe plusieurs découpages de séquence qui mettent en avant différents aspects des données.

- Créer des séquences de n actions.
- Créer des séquences de k minutes.
- Créer des séquences calquées sur une phase de possession.

Dans notre cas, nous créerons les séquences en fonction de la possession de la balle : de la récupération de la balle par l'équipe jusqu'à sa perte ou bien un essai (et transformation).

Pour ce faire, nous avons utilisé l'action Possession déjà présente. En effet, cette action nous renseigne dans les colonnes *ps_timestamp* et *ps_endstamp* le début et la fin d'une phase de possession de balle.

Note : Nous avons créés les datasets respectifs de Toulouse et Leinster à partir de cette méthode. Ainsi, les actions défensives que l'on retrouvera dans *data_2300* seront les actions réalisées par les joueurs toulousains. Et vice-versa.

2.4 Représentation des séquences

Afin de pouvoir effectuer une analyse par clustering, nous avons dû représenter nos séquences de façon à introduire une notion de distance entre elles (voir 3.1). Nous avons donc décidé d'utiliser des chaînes de caractères en tant que représentation de séquences.

Comme nos séquences sont par essence des listes d'actions, nous avons associé à chaque action une lettre. Nous avons transformé alors nos séquences en listes de lettres pour finalement les convertir en chaînes de caractères en concaténant leurs éléments.

Ainsi, une séquence qui originellement ressemblait à ça...

[1] "Restart"	"Collection"	"Tackle"	"Tackle"	"Carry"	"Ruck"	"Ruck 00A"	"Ruck 00A"
[9] "Ruck 00A"	"Ruck 00A"	"Collection"	"Kick"	"Defensive Exit"	"Possession"		

...devient la chaîne de caractères suivante :

[1] "ABCCDEFFFFBGHI"

Cette approche nous permet de stocker nos séquences transformées dans des listes, afin de les retrouver sous leur forme d'origine. En effet, nous pouvons ainsi accéder aux différents attributs qui seraient perdus si on essaie de reconstruire nos séquences à partir de leur état transformé (sous forme de chaîne de caractères).

Par exemple, il est impossible de récupérer les *timestamps* des actions à partir de la chaîne "ABCCDEFFFFBGHI", mais en stockant nos séquences sous état transformé (sous forme de chaîne de caractères) dans une liste, on peut retrouver la séquence sous forme d'origine qui correspond à la séquence transformée en regardant son indice dans la liste.

3 Clustering de séquence

Après avoir construit nos séquences et les avoir représentées convenablement, nous allons les utiliser pour faire du clustering. En effet, vu le grand nombre de séquences qu'on va manipuler, les graphes qu'on obtient avec BupaR sont peu lisibles (voir Figure 13). L'objectif est de trouver des clusters et leurs représentants qui seront utilisés comme nouveau dataset.

Cependant, vu que nos données sont catégorielles et non pas quantitatives, nous ne pouvons pas utiliser une distance usuelle (comme par exemple la distance euclidienne). Nous allons donc définir notre propre distance afin de pouvoir comparer nos séquences entre elles.

3.1 Distance

Commençons par définir les propriétés d'une distance qui sont nécessaires pour le CAH.

Soit d une distance d'un espace E :

- $d : E \times E \longrightarrow \mathbb{R}^+$
- $\forall a, b \in E, d(a, b) = 0 \Leftrightarrow a = b$
- Symmétrie : $\forall a, b \in E, d(a, b) = d(b, a)$
- Inégalité triangulaire : $\forall a, b, c \in E, d(a, b) \leq d(a, c) + d(c, b)$

3.1.1 Caractéristiques

La distance étant une métrique de similarité, des distances variables entre lettres equivaut à une ressemblance variable entre les actions.

On pourrait attribuer un nombre arbitraire à chaque action et calculer des distances, centres à partir de ces nombres. Ces nombres seraient choisis en fonction de la proximité des actions (les actes défensifs proches par exemple). Néanmoins, faire ces choix n'est pas toujours évident et fait rentrer en paramètre d'analyse notre interprétation des actions. Ce qui peut fausser les résultats.

Ainsi, l'équidistance des lettres nous semble être imposée.

Deuxième critère : l'ordre. En effet, puisque nous avons pour objectif d'analyser les actions en tant que processus. La distance choisie doit prendre en compte l'ordre des actions. Par conséquent, $d(AC, CA) \neq 0$ et même mieux $d(AC, DE) \simeq d(AC, CA)$, dans la mesure où les séquences AC et CA serait aussi différente qu'avec des lettres distinctes.

Ainsi, le choix de la distance se fera sur les caractéristiques suivantes :

- Distance uniforme entre les lettres.
- Importance de l'ordre des lettres.

En informatique, pour mesurer la similarité entre deux chaînes, il est coutume d'utiliser des distances d'édition appelée également *edit distance*. Ces distances comptent le nombre d'opérations nécessaire pour passer d'une chaîne à l'autre. Cette catégorie regroupe notamment la distance de Levenshtein, de Hamming mais surtout la distance LCS que l'on utilisera.

3.1.2 Distance de Levenshtein

Intéressons nous d'abord à la distance de Levenshtein. Celle-ci compte le nombre de suppressions, insertions et substitutions de lettres à faire pour lier les deux chaînes.

Exemple : Pour CAFE et THE, il faut 2 substitutions ($F \rightarrow H$, $A \rightarrow T$) ainsi qu'une suppression du C. On obtient ainsi une similitude de 3.

Pour calculer cette distance systématiquement, il existe une méthode utilisant une matrice $n \times m$ (avec n, m les longueurs respectives des deux mots). Notons alors que l'algorithme est de complexité polynomiale. Cette méthode sera utilisée pour la distance LCS qui est une variante de cette distance..

3.1.3 Distance LCS

La distance LCS pour *Longest Common Subsequence* compte le nombre d'élément non compris dans la plus longue sous-séquence commune. Cette distance est adéquate puisqu'elle prend en compte l'ordre des lettres avec la sous-séquence. Et c'est une distance d'édition : les lettres sont équidistantes.

Exemple :

Pour BONJOUR et BONSOIR. La plus longue sous-séquence commune est BONOR. Ainsi, dans BONJOUR, J et U n'appartiennent pas à la sous-séquence, de même pour S et I dans BONSOIR. On en conclut que $d(\text{BONJOUR}, \text{BONSOIR}) = 2 + 2 = 4$

Au lieu de calculer directement la sous-séquence commune entre les deux mots, une autre méthode s'appuie sur le calcul de la distance de Levenshtein. La distance LCS correspond à un cas particulier de la distance de Levenshtein où seules les insertions et suppressions de lettres sont autorisées.

Algorithme :

L'algorithme se base sur un tableau à deux dimensions, dont les dimensions correspondent aux longueurs des deux chaînes de caractères. Le tableau est rempli de manière itérative, en suivant les règles suivantes :

- La première ligne et la première colonne du tableau sont initialisées respectivement à 0, 1, 2, ..., jusqu'à la longueur de la première ou de la deuxième chaîne. Cela représente le nombre d'opérations nécessaires pour transformer une chaîne vide en l'autre.
- Pour les autres cases du tableau, on calcule la distance en fonction des deux cas possibles :
 - Si les caractères des deux chaînes à la position courante sont identiques, la distance est égale à la valeur de la case diagonale en haut à gauche.
 - Sinon, la distance est égale à 1 + le minimum entre les deux valeurs suivantes :
 - La distance de la case précédente dans la même colonne (suppression du caractère dans la première chaîne).

- La distance de la case précédente dans la ligne précédente (insertion du caractère dans la première chaîne).

À noter que cette méthode est la même que pour la distance de Leveinshtein, si ce n'est que l'on a interdit l'opération de remplacement qui est en diagonale en haut à gauche de la case courante.

Remplacement +1	Suppression +1
Insertion +1	position courante

En remplissant le tableau de cette manière, on obtient la distance LCS entre les deux chaînes dans la case en bas à droite du tableau.

Pour illustrer la méthode, nous allons calculer $LCS(CHIEN, NICHE)$.

	« »	C	H	I	E	N
« »	0	1	2	3	4	5
N	1					
I	2					
C	3					
H	4					
E	5					

Itération 0

	« »	C	H	I	E	N
« »	0	1	2	3	4	5
N	1	2				
I	2					
C	3					
H	4					
E	5					

Itération 1 : $N \neq C \rightarrow 1+1$

	« »	C	H	I	E	N
« »	0	1	2	3	4	5
N	1	2	3			
I	2					
C	3					
H	4					
E	5					

Itération 2 : $N \neq H \rightarrow 2+1$

	« »	C	H	I	E	N
« »	0	1	2	3	4	5
N	1	2	3	4	5	4
I	2					
C	3					
H	4					
E	5					

Itération 5 : $N = N \rightarrow 4+0$

$\Rightarrow *$

	« »	C	H	I	E	N
« »	0	1	2	3	4	5
N	1	2	3	4	5	4
I	2	3	4	3	4	5
C	3	2	3	4	5	6
H	4	3	2	3	4	5
E	5	4	3	4	3	4

Etat final

On en déduit que $LCS(CHIEN, NICHE)=4$.

3.2 Classification par Ascendant Hiérarchique

Il existe de nombreuses techniques statistiques visant à partitionner une population en différentes classes ou sous-groupes. La classification ascendante hiérarchique (CAH) est l'une d'entre elles. On cherche à ce que les individus regroupés au sein d'une même classe (homogénéité intra-classe) soient le plus semblables possibles tandis que les classes soient le plus dissemblables (hétérogénéité inter-classe).

Un des avantages du CAH est qu'on travaille à partir des dissimilarités entre les individus que l'on veut regrouper. On peut donc choisir un type de dissimilarité adapté au sujet étudié et à la nature des données.

L'un des résultats est le dendrogramme (voir 2), qui permet de visualiser le regroupement progressif des données. On peut alors se faire une idée d'un nombre adéquat de classes dans lesquelles les données peuvent être regroupées.

3.2.1 Principe du CAH

Le principe de la CAH est de rassembler des individus selon un critère de ressemblance défini au préalable qui s'exprimera sous la forme d'une matrice de distances, exprimant la distance existant entre chaque individu pris deux à deux. Deux observations identiques auront une distance nulle. Plus les deux observations seront dissemblables, plus la distance sera importante. La CAH va ensuite rassembler les individus de manière itérative afin de produire un dendrogramme ou arbre de classification.

La classification est ascendante car elle part d'une situation où tous les individus sont seuls dans une classe, puis sont rassemblés en classes de plus en plus grandes. Elle est hiérarchique car elle produit une hiérarchie H , l'ensemble des classes à toutes les étapes de l'algorithme, qui vérifie les propriétés suivantes :

- $\Omega \in H$: au sommet de la hiérarchie, lorsqu'on groupe de manière à obtenir une seule classe, tous les individus sont regroupés.
- $\forall \omega \in \Omega, \{\omega\} \in H$: en bas de la hiérarchie, tous les individus ω se trouvent seuls.
- $\forall (h, h') \in H, h \cap h' = \emptyset$ ou $h \subset h'$ ou $h' \subset h$: si l'on considère deux classes du regroupement, alors soit elles n'ont pas d'individu en commun, soit l'une est incluse dans l'autre.

3.2.2 Algorithme

Voici le principe du CAH :

1. On commence à calculer la dissimilarité entre tous nos objets.
2. On regroupe dans une classe les deux objets ayant la dissimilarité plus petite.
3. On calcule ensuite la dissimilarité entre cette classe et le reste des objets.
4. On itère jusqu'à que tous les objets soient regroupés.

Ces regroupements successifs produisent un arbre binaire de classification (dendrogramme), dont la racine correspond à la classe regroupant l'ensemble des objets. Ce dendrogramme représente une hiérarchie de partitions. On peut alors choisir une partition en tronquant l'arbre à un niveau donné.

Dans notre cas, nous n'avons pas dû implémenter l'algorithme puisqu'une fonction existe déjà dans R : *hclust()*. Cependant, il est nécessaire de spécifier la méthode utilisée pour regrouper les classes. Nous avons donc utilisé la méthode de Ward.

3.2.3 Exemple et dendrogramme

Voici un dendrogramme obtenu à partir de l'application du CAH à nos séquences :

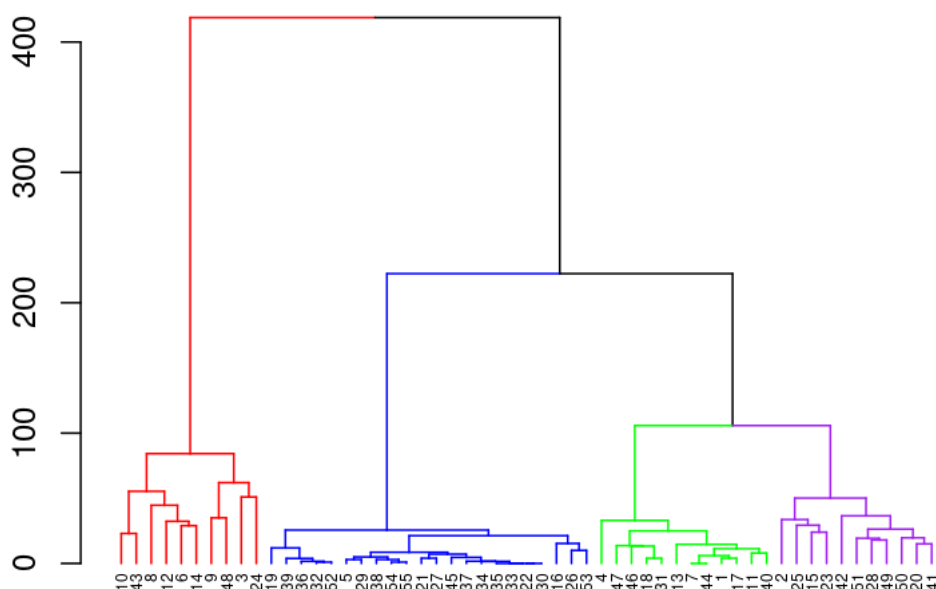


FIGURE 2 – Dendrogramme des séquences

3.3 Elbow Method

La méthode du coude ou *elbow method* est une technique empirique utilisée pour déterminer le nombre optimal de clusters dans un algorithme de clustering de classification.

Le nombre de cluster est un paramètre important qui a un réel impact sur la classification.

Si le nombre de cluster est trop faible, on risque d'avoir des groupes avec des éléments avec une forte dissimilarité, pour lesquels on devrait avoir au moins deux groupes. A l'inverse, si le nombre de cluster est trop élevé, on risque d'avoir des groupes trop semblables. On pourrait alors fusionner ses deux groupes sans perte d'information élevée. De plus, dans notre cas où on utilise la classification pour réduire et synthétiser le nombre de séquences à analyser, cela serait contre-productif.

L'*elbow method* consiste à analyser la variation de la somme des variances intra-classes en fonction du nombre de clusters k . On représente graphiquement ces variances en fonction de k , et on observe la forme de la courbe obtenue.

Si la courbe décroît fortement jusqu'à un certain k puis s'atténue, le k en question est considéré comme le nombre de cluster idéal.

Si la courbe décroît de manière régulière, on ne peut pas désigner de k avec cette méthode.

Dans l'exemple à droite, le rang k est entouré en rouge.

Dans cet exemple tiré de nos données, la décroissance est régulière excepté pour $k=2$.

Deux clusters n'étant pas assez, nous avons opté pour 4 clusters pour le dataset de Toulouse et 5 pour celui de Leinster.

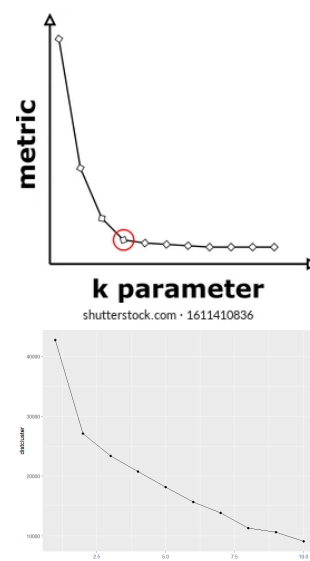


FIGURE 3 – Courbe pour les phases offensives de Leinster

Cette méthode est facile à mettre en œuvre et permet d'identifier intuitivement le nombre optimal de clusters.

Cependant, la méthode n'est pas d'une grande rigueur car l'interprétation reste subjective. L'échelle des axes peut également modifier la perception de la courbe et donc notre analyse. Dans notre cas, l'objectif n'étant pas de classer les séquences mais de réduire leur nombre en limitant la perte de précision. Ce pourquoi nous avons choisi ce nombre assez librement.

3.4 Représentant d'un cluster

Une fois les clusters formés, nous devons désigner un élément représentatif pour chaque cluster. A noter qu'il peut appartenir ou non au cluster. Un élément représentatif est par définition l'élément le plus semblable. Pour cela, nous avons deux options :

- Nous calculons/créons le représentant comme étant le "centre" du cluster, on parle alors de centroïde.
- Nous trouvons l'élément qui a le moins de dissimilarité avec les autres éléments du cluster, on parle alors de médoïde.

3.4.1 Représentant-centroïde

Cette option est difficilement applicable dans notre cas. Calculer le centre comme l'on ferait pour un groupe de point est impossible ici. De par la construction de notre espace, il n'existe pas de milieu entre deux séquences.

Néanmoins, nous pouvons tout de même créer ce qui s'apparenterait au centre du cluster. Pour ce faire, nous appliquons un alignement des lettres basé sur la sous-séquence commune et faisons un tirage au sort sur les autres lettres.

Exemple : Création du milieu entre BONJOUR, BONSOIR, JAMBON.

1) Calcul de la sous-séquence commune.

Ici, il s'agit de BON : **B**ONJOUR, **B**ONSOIR, JAMBON.

2) Alignement des lettres.

			B	O	N	J	O	U	R
			B	O	N	S	O	I	R
J	A	M	B	O	N				

3) Tirage des lettres.

Le tirage se fait par colonne. On peut donc comparer des caractères avec du vide. A la fin, si un espace se trouve au milieu de la séquence, on le supprime.

Supposons que l'on obtienne le tirage suivant :

__MBON__OIR. → MBONOIR.

Pourquoi cette méthode est cohérente ?

Tout d'abord, la sous-séquence commune (l'information principale) reste intacte. Ce qui nous permet d'avoir un centre qui garde l'information utile .

De plus, avec le tirage uniforme, les lettres les plus présentes ont le plus de chance d'apparaître dans le centre final. Ce qui est en adéquation avec une dissimilarité basse.

Néanmoins cette méthode a ses inconvénients qui nous poussent à l'abandonner.

Pour commencer, la sous-séquence commune tend à rapetisser lorsque la taille des clusters augmente. En effet, bien qu'ils soient choisis sur un critère de similarité, les éléments au sein d'un cluster restent différents. Cela a pour conséquence de mettre l'accent sur le tirage au sort qui est originellement restreint aux lettres non communes, mais qui s'étend lorsque la taille de la séquence commune diminue. De plus, calculer la sous-séquence commune, au-delà de 2 chaînes, est relativement fastidieux. Les algorithmes mis à notre disposition n'assurent pas une réponse optimale ni même une réponse tout court. L'aspect aléatoire est également un inconvénient. On obtient des résultats différents et donc nous n'avons pas toujours la même analyse. Ce qui renforce la difficulté de la tâche. Pour terminer, cette méthode qui de prime à bord semble conserver l'ordre des actions, crée des incohérences notamment dans les zones de tirage. Ce qui crée des centroïdes qui ne ressemblent plus aux éléments du cluster.

3.4.2 Représentant-médoïd

Nous avons donc choisi la deuxième méthode : les médoïdes. On va donc chercher l'élément du cluster dont la dissimilarité moyenne est la plus basse.

Pour déterminer la dissimilarité moyenne, on réutilise la matrice de distance déjà établie et calculons la somme des distances au carré. C'est-à-dire, pour un élément x du cluster A : $dismoy(x) = \sum_{y \in A} dist(x, y)^2$.

Cette métrique est pertinente car elle utilise de manière sous-jacente la MSE. $MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = (\mathbb{E}[\hat{\theta}] - \theta)^2 + Var(\hat{\theta})$

Dans notre cas, le $\hat{\theta}$ représente la moyenne des distances, et le $\theta = 0$. Cela reste cohérent dans la mesure où, bien qu'un élément de dissimilarité nulle avec tous les autres n'existe pas (sauf cluster à un élément), celui dont la MSE est minimale est bien le meilleur candidat. De plus, la MSE s'exprimant également comme la somme du biais et de la variance, nous confirme la légitimité du représentant, en évitant des valeurs absurdes.

```
1  #INPUT: matrice de distance du cluster, vecteur des éléments du cluster
2  #OUTPUT: l'élément représentant
3  find_representant <- function(matrix_dist, vec_str){
4      row_dist = rowSums(matrix_dist**2) #somme les valeurs de chaque case au carré
5      ranked_dist<-min_rank(row_dist) #classe les elements par ordre croissant
6      return(vec_str[which(ranked_dist==1)]) #retourne l'élément en 1ère position
7  }
```

4 Analyse

Maintenant que nous avons introduit les données ainsi que défini les outils dont nous aurons besoin, passons à l'analyse.

Pour donner un sens à l'analyse de nos données, nous devons les orienter dans le sens d'une problématique. Dans notre cas, nous voulons comprendre ce qui a fait la différence entre le Stade Toulousain et l'équipe de Leinster lors de ce match, où la deuxième équipe est sortie largement gagnante.

Pour ce faire, nous commencerons par mettre en avant les résultats d'indicateurs de performances et voir s'ils coïncident avec la victoire de Leinster. On en profitera, si les données le permettent, pour isoler des tendances de jeux divergentes. Ensuite, nous examinerons les séquences d'actions qui nous offrent un point de vu plus réaliste.

4.1 Analyse générale

Dans cette section, nous allons traiter différents indicateurs de performance. Nous aborderons des facteurs comme la possession, la proportion des actions et le gain de distance. De plus, en se penchant sur l'évolution du score durant le match, il y a une période critique de domination de Leinster sur Toulouse (voir Figure 14). Cette période s'étend de la 10^{ème} à la 30^{ème} minute, durant laquelle 24 points y ont été inscrits. On s'intéressera ponctuellement aux données sur cette période.

4.1.1 Possession

La possession de la balle est un indicateur primordial pour analyser la dynamique d'un match. Une équipe qui a la balle, c'est une équipe qui peut marquer mais aussi une équipe qui n'est pas attaquée.

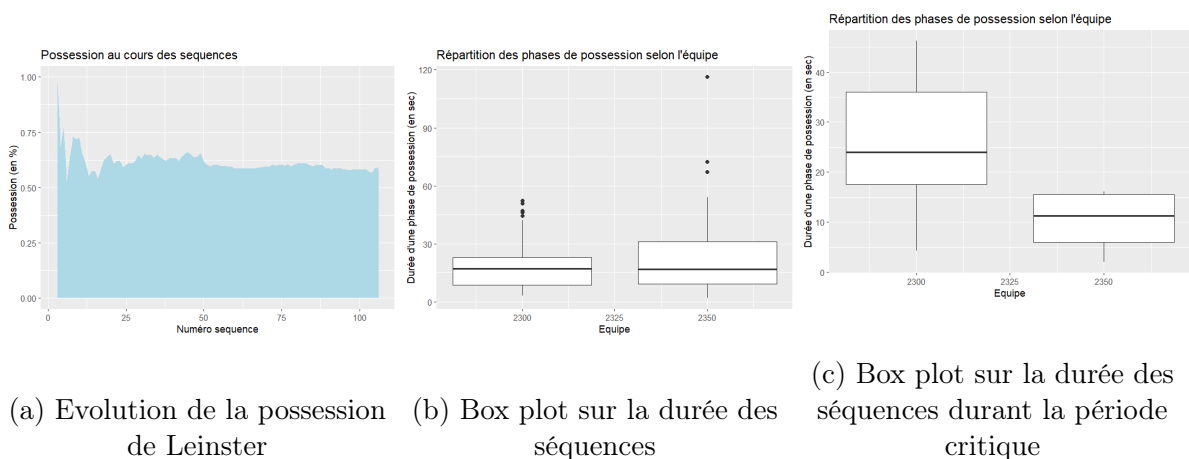


FIGURE 4

Commençons par analyser l'évolution de la possession au cours du match. On constate une forte variation jusqu'à la 50^{ème} séquence puis la possession se stabilise autour de 60-40 pour Leinster. Cette variation correspond à la période de la 1^{ère} mi-temps, où Leinster a pris plus de risques, marqués de points. La seconde phase, elle, s'expliquerait par un jeu

moins dangereux pour conserver son avance.

On peut également s'intéresser à la durée d'une phase de possession. Dans la mesure où une équipe qui arrive à garder la balle plus longtemps contrôle le jeu.

Le graphique nous indique qu'il n'y a probablement pas de différence sur la durée moyenne de possession. Cependant, si on se penche sur la période isolée, Leinster (2300) a des phases de possession significativement plus longues (voir Figure 4).

On en conclut que la domination de Leinster sur Toulouse est bien retranscrite dans la possession de la balle. Ce qui nous laisse penser que Leinster a eu plus de facilités à contrôler le match.

4.1.2 Les actions

Nous allons interpréter des données concernant les actions : leur fréquence d'apparition dans des séquences et leur position sur le terrain.

Il va sans dire que analyser les actions directement est essentiel pour notre étude.

Nous débutons avec le pourcentage de séquence dans lequel apparaît une action. On comptabilise donc une seule fois l'action par séquence.

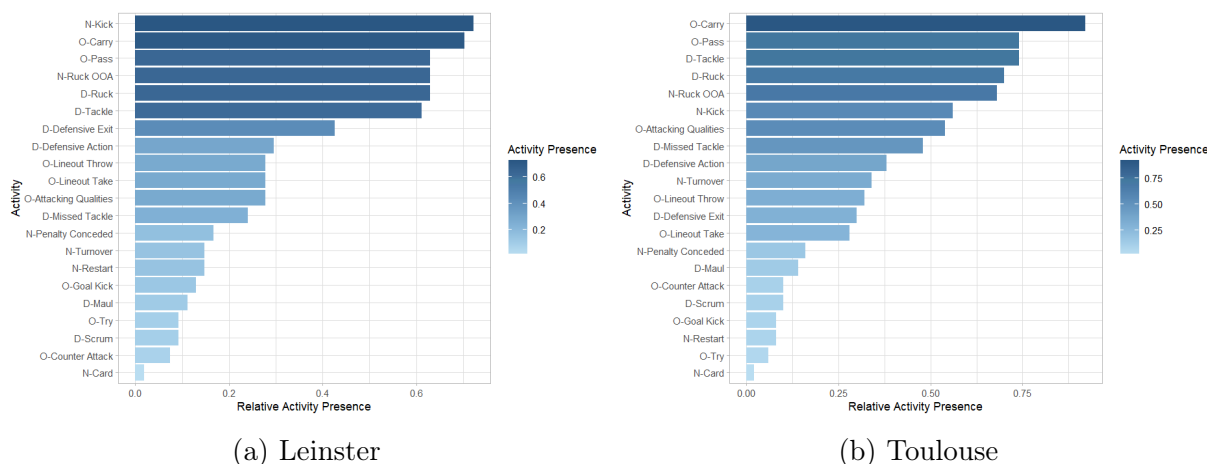


FIGURE 5 – Fréquence des séquences contenant une action

Les 6 actions en tête sont identiques pour les deux équipes mais l'ordre et l'échelle sont distincts.

Leinster utilise globalement plus souvent le coup de pied dans ses phases de possession. L'utilisation du jeu au pied est considérée comme bénéfique pour la victoire dans le monde du rugby.

Les séquences toulousaines quant à elles comportent plus souvent des courses et des passes, tout en se confrontant plus à la défense adverse.

Maintenant, ajoutons à notre analyse le nombre d'apparitions de chaque action, avec répétitions (voir Figure 19). La confrontation de Toulouse à la défense de Leinster est largement mise en avant. Cette dernière a effectué plus de plaquages, dans la plupart des séquences adverses alors qu'ils étaient moins souvent en danger (40% de possession toulousaine).

Note : Ce dernier graphique laisse entendre qu'il existe des actions plus importantes que d'autres, indépendamment de l'équipe. Les actions de fondamentales seraient le Ruck (avec Ruck OOA), Tackle, Pass et Carry, les autres étant de second plan. Cette notion sera utilisée par la suite

Pour terminer, on va se pencher sur la zone de jeu. Le terrain a été divisé en 3 zones :

- les 22 derniers mètres du camp de Toulouse.
- la zone centrale du terrain.
- les 22 derniers mètres du camp de Leinster.

Donc lorsque des actions se jouent dans les 22 derniers mètres, l'équipe qui défend est en danger car proche de sa ligne de but.

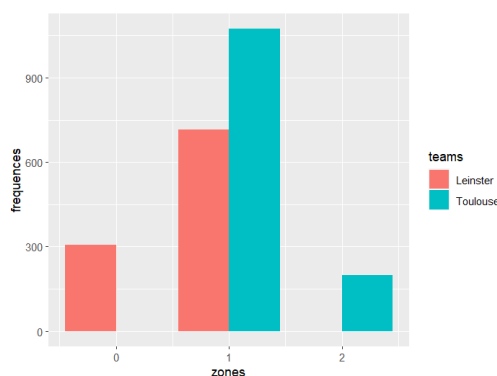


FIGURE 6 – Nombre d'actions offensives par équipe selon la zone de jeu

Tout d'abord, on constate que l'équipe de Toulouse a plus d'actions que Leinster, bien que cette dernière ait une possession de la balle plus importante.

D'autre part, on voit que sur les phases offensives de Leinster, les actions ont plus souvent lieu dans les 22 mètres adverses. Cette dernière donnée est indicatrice de la pression de Leinster sur Toulouse.

4.1.3 Gain de distance

Pour essayer de quantifier l'efficacité des équipes, nous allons nous pencher sur les distances gagnées en moyenne quand une équipe a la balle.

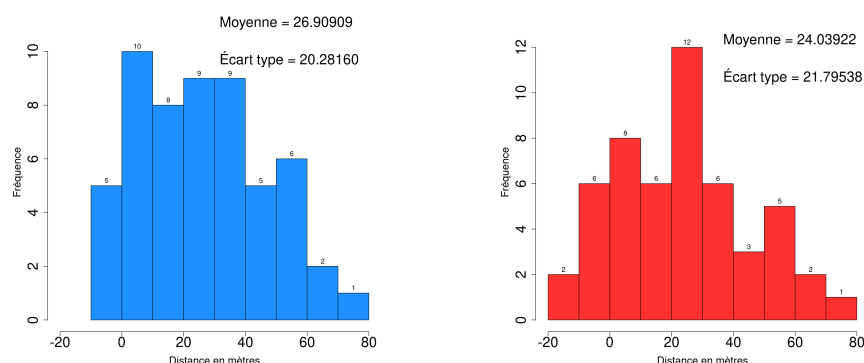


FIGURE 7 – Histogrammes des distances gagnées par les deux équipes

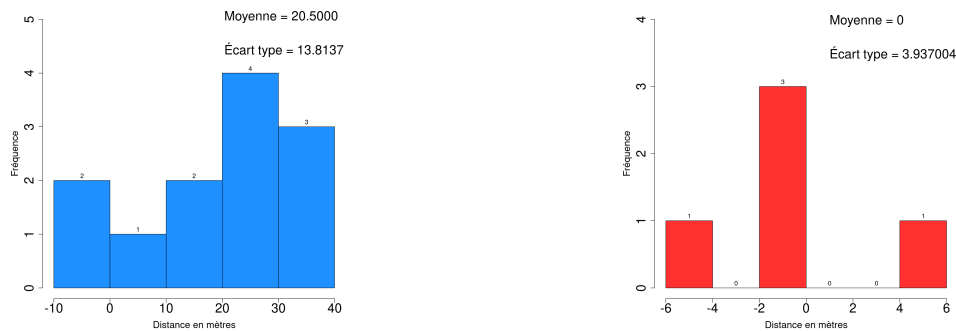


FIGURE 8 – Histogrammes des distances gagnées par les deux équipes entre la 10ème et 30ème minute

En comparant les moyennes, on voit que le Leinster gagne plus de distance que le Stade Toulousain.

Cependant, cette différence n'est pas très significative et on peut s'en convaincre avec un test non apparié de comparaison de moyennes (voir figure 17 en annexe). De plus, si on tient compte que le Carry (selon la figure 19) est très souvent présent quand Toulouse a la possession (environ 90%), on peut supposer que leur Carry est moins efficace que celui du Leinster, présent dans 70% des cas où ils ont la possession.

Regardons maintenant les distances gagnées en moyenne par les équipes pendant la période critique. Ici on voit que l'écart de gain est réellement significatif, ce qui peut être à nouveau confirmé avec un test de comparaison de moyennes (voir figure 18 en annexe). Cet écart peut s'expliquer par la durée des séquences relatives aux deux équipes (voir figure 4) puisque au rugby on se déplace sur toute la durée de possession de balle. Contre toute attente, la moyenne des distances gagnées par Leinster est inférieure à sa moyenne générale (20 mètres contre 26).

Cette information nous apprend que ce n'est pas la distance gagnée qui importe, mais la distance gagnée par rapport à celle de l'adversaire. On constate bien que le Stade Toulousain n'a pas pu développer son jeu vu que son gain de distance est en moyenne nul.

4.2 Analyse de processus (post-clustering)

Pour analyser les processus, nous avons opté pour la bibliothèque BupaR. Spécialisée pour l'étude de processus commerciaux, nous appliquerons certains de ses outils à notre étude. Pour le clustering de nos données, nous utilisons la méthode CAH décrite précédemment, ainsi que les métriques associées. Dans l'objectif d'avoir des données lisibles (graphiquement).

4.2.1 Creation des événements dans bupaR

Les eventlog sont générés à partir des dataframes, chaque ligne représentant un événement distinct (ici des actions). Dans le dataframe, chaque événement est associé à :

- l'identifiant de l'action : nous prendrons son nom pour des raisons d'esthétique.
- un identifiant de la séquence dans laquelle s'est produite l'action : `sequence_id`.
- un cycle de vie : timestamp de début et timestamp de fin de l'événement au format `ymd_hms` que l'on obtient à partir des formats POSIX.

```

1 casted_data<-convert_timestamps(casted_data, columns = c("start", "complete"),
2   format = ymd_hms) #start et complete <=> timestamp et endstamp
3 casted_data_log<-activitylog(casted_data, case_id = "sequence_id",
4   activity_id = "actionName", timestamps = c("start", "complete"),
5   resource_id = "sequence_id")

```

À partir de maintenant, on peut créer un eventlog à partir d'un dataframe quelconque. Le dataframe en question peut être celui des données sans clustering, auquel cas on peut directement créer les eventlogs. Mais, si on souhaite le faire à partir des représentants de cluster, il reste du travail à faire.

Dans le cas échéant, il faut construire le dataframe à partir des représentants. On transforme les chaînes de caractères en séquences, et on ajoute les séquences dans un même dataframe, à la suite. Cependant, il est primordial de pondérer chaque représentant par le nombre d'éléments qu'il représente. Ainsi, on va ajouter autant de fois le représentant qu'il y a d'élément dans son cluster.

Attention, il faut veiller à modifier son identifiant de séquence. On opte pour incrémenter de 200 à chaque ajout pour éviter des collisions d'identifiants avec d'autres séquences : $i=1, 202$; $i=2, 402$; $i=3, 602$...

```

1 df_representants <- c()
2 #On ajoute les représentants dans un vecteur de manière pondérée
3 for (i in 1:N_CLUSTER){
4   seq_i <- str_to_seq(nodes[i]) #on transforme le représentant en sequence
5   for (j in 1:get_size_cluster(df_clustered, i)){
6     df_representants<-rbind(df_representants,
7       mutate(seq_i, sequence_id=sequence_id+200*j)) #ajout représentant
8   }
9 }

```

4.2.2 Matrice de précédence

La matrice de précédence est la matrice d'adjacence du graphe qui représente les séquences (nous verrons ces graphes dans la section suivante). Elle permet de visualiser les relations directes entre les différentes actions.

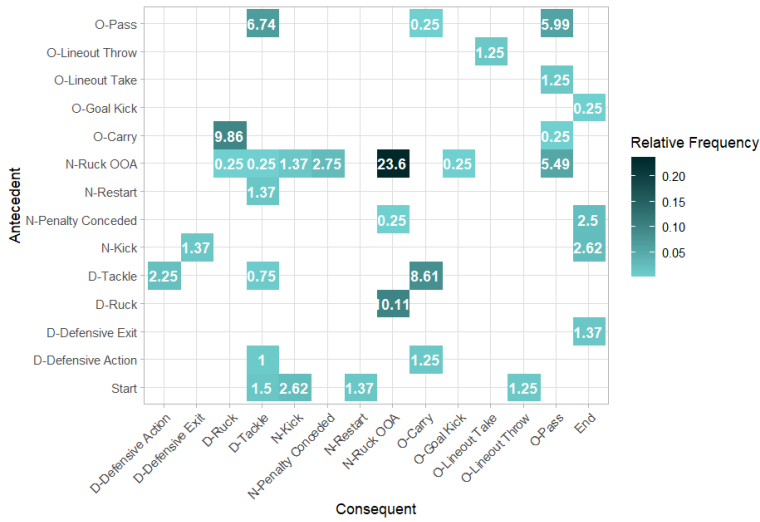


FIGURE 9 – Matrice de précedence des actions de Leinster

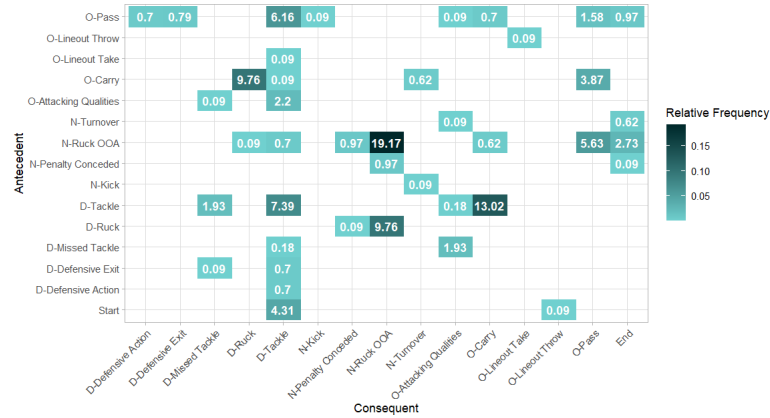


FIGURE 10 – Matrice de précedence des actions de Stade Toulousain

La première remarque est la différence des actions représentées. On ne retrouve pas les mêmes actions dans la matrice de Leinster et Toulouse. Cependant, ce n'est pas une contrainte dans la mesure où les actions fondamentales (Pass, Tackle, Ruck, Carry) apparaissent dans les deux tableaux. Les actions différentes sont les actions de second plan.

Deuxièmement, les relations les plus fortes apparaissent dans les deux matrices. Comme Ruck OOA → Ruck OOA ou Tackle → Carry, par exemple. On peut en déduire que ces tendances en commun sont inhérentes au rugby.

Néanmoins, certaines relations sont plus marquées chez l'un que l'autre.

Sur l'aspect défensif, on constate que la relation Tackle → Tackle est significativement plus encrée pour Toulouse que pour Leinster, respectivement 7.39 contre 0.75. Sachant qu'il s'agit des plaquages effectués par Leinster sur les joueurs toulousains, on peut en déduire que l'équipe enchaîne plus souvent les plaquages. Et donc, réussi à être plus percutante défensivement.

On s'aperçoit aussi que le nombre de passes suivies par une autre passe est beaucoup plus haut pour Leinster (5.99) que pour Toulouse (1.58). Ceci témoigne d'une aptitude à faire circuler la balle plus aisément : un vrai atout offensif.

4.2.3 Graphe de processus

Un graphe de processus est un graphe orienté où chaque activité est représentée par un noeud, et chaque relation entre les activités est indiquée par des arcs.

La valeur inscrite dans le noeud représente la fréquence d'apparition de l'action. La valeur sur l'arc indique le pourcentage de fois où l'action au départ de la flèche a précédé celle au bout. Le graphe de processus nous donne une vision globale des séquences : les relations directes de la matrice de précedence, des relations indirectes (sur 2-3 arcs de longueur) et également la proportion de chaque action.

Cependant, nous travaillons sur des représentants de cluster sélectionnés sur l'ordre des actions exclusivement. Ainsi, les informations de proportion d'action ne sont pas utilisables rigoureusement (bien que fortuitement correctes).

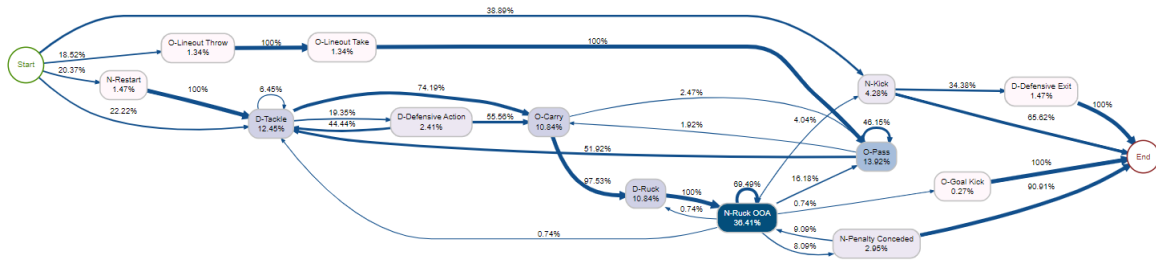


FIGURE 11 – Graphe de processus pour Leinster

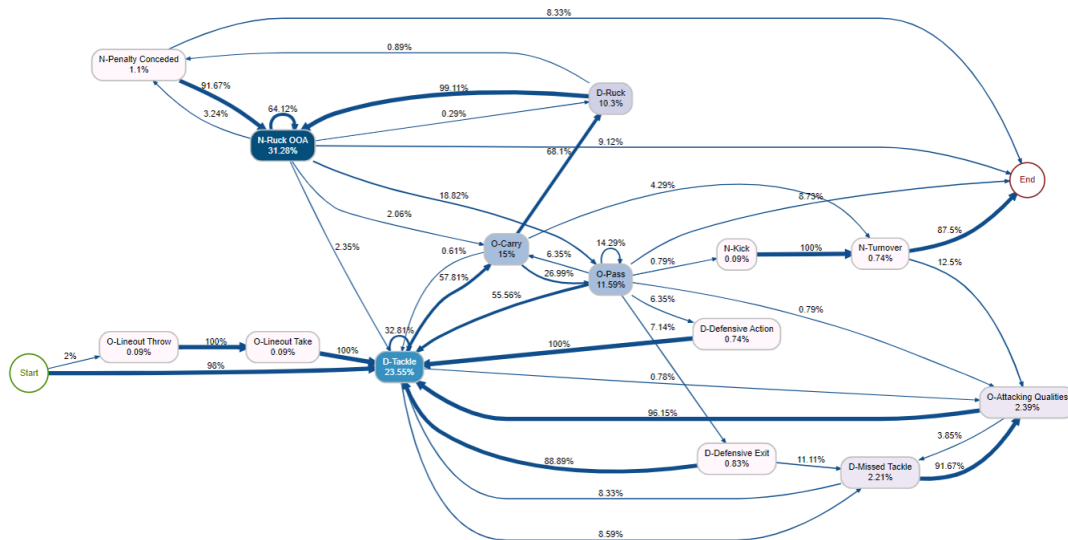


FIGURE 12 – Graphe de processus pour Toulouse

Nous comparerons les deux graphes en décomposant les séquences en 3 parties :

- Début de la séquence

Dans le graphe concernant Toulouse, on observe que les séquences commencent par un plaquage adverse. Cette information confirme l'hypothèse selon laquelle Leinster fait un pressing plus agressif. Tandis que Leinster arrive à rentrer dans l'action avec des passes, notamment sur les touches.

- Milieu de la séquence

On constate que les liens entre les actions fondamentales (Tackle, Carry, Ruck, Pass) sont presque identiques chez les deux équipes. En effet, ces actions de par leur importance et leur lien fort, forment un cycle qui concentre le jeu. Le reste des actions vont "graviter" autour d'elles.

- Fin de la séquence

Les deux équipes perdent la balle pour des causes distinctes. La possession du Stade Toulousain s'arrête de plusieurs façons subies (Penalty Conceded, Ruck-OOA, Pass, Turnover). Bien que Leinster perde la balle aussi à cause de Penalty Conceded, ils font principalement des dégagements au pied (Kick) afin d'éloigner le danger.

On en conclut que la partie ventrale de la séquence est sensiblement identique pour les deux équipes. La différence est surtout présente en début et en fin de séquence : c'est là que Leinster se démarque.

4.3 Synthèse

L'analyse du match entre le Stade Toulousain et le Leinster met en évidence plusieurs facteurs clés pouvant expliquant la victoire de Leinster.

Le pressing défensif de Leinster a grandement participé à la victoire. Celui-ci se caractérise par le nombre important de Tackles, leur fréquence élevée et leur apparition quasi-systématique dans les phases offensives adverses.

Offensivement, l'équipe s'est surtout démarquée durant une période de 20 minutes au début du match. C'est là qu'elle a creusé l'écart avec des phases de possession plus longues qui ont permis de gagner du terrain. Le tout appuyé par un jeu au pied et la main plus fluide.

Néanmoins, l'analyse de processus nous dit que les tactiques utilisées par les deux équipes ne sont pas aussi distinctes comme on voudrait le penser. Mais Leinster a su faire la différence sur des détails à priori importants.

En résumé, la victoire de Leinster résulte de leur possession de balle supérieure, leur stratégie offensive efficace, une pression constante sur l'adversaire avec une gestion du match maîtrisée.

5 Conclusion

Le création de processus est plutôt naturelle dans ce contexte car elle se calque sur la possession de balle de l'équipe. Elle nous a permis de structurer notre dataset et de comparer ses éléments entre eux : soit les séquences entre elles, soit en créant de nouveaux datasets à partir de ces séquences.

L'action Possession, spécifique à ce dataset, permettait de faire une analyse générale sans créer des séquences. Dans un autre contexte, nous aurions dû passer par la création de processus, pour calculer nous même la durée et les distances de chaque séquence.

De plus, la manipulation de séquences est très développée (grâce par exemple à des librairies comme BupaR), ce qui nous offre un grand éventail de modélisations graphiques nous permettant de mieux visualiser les interactions.

Cependant, si le nombre de séquences est élevé, il est peut-être nécessaire de faire du clustering afin d'obtenir des résultats plus visibles. Si nos données sont de type catégorielles, il n'est pas possible d'utiliser des distances usuelles. Il faut alors définir une distance propre qui peut ne pas être optimale, entraînant alors une perte de précision lors du clustering.

Analyser de façon si générale est assez compliqué, il aurait été plus intéressant se focaliser sur des processus simples (avec un nombre d'actions réduit) pour ne pas faire un clustering.

D'autre part, l'utilisation d'outil fourni par BupaR comme les graphes de processus, peuvent difficilement être analysés seuls. Dans notre cas, nous avons dû compléter l'étude de processus par une analyse dite "classique".

Pour terminer, nous avons fait le choix de se concentrer sur l'équipe dans sa globalité. Mais il serait pertinent de prendre en compte les données des joueurs, pour compléter une étude sportive comme celle-ci. BupaR nous permet de renseigner ce genre d'information dans l'eventlog et donc d'examiner le rôle de chaque joueur au sein d'un processus.

6 Bibliographie

Lien du repos github : <https://github.com/jacopoli/StageL3MIDL.git>

- <https://medium.com/@ethannam/understanding-the-levenshtein-distance-equation-for-beginners-c4285a5604f0>
- <https://www.xlstat.com/fr/solutions/fonctionnalites/classification-ascendante-hierarchique-cah>
- <https://larmarange.github.io/analyse-R/classification-ascendante-hierarchique.html>
- https://fr.wikipedia.org/wiki/Regroupement_hierarchique
- <https://datascientest.com/machine-learning-clustering-focus-sur-algorithme-cah>
- <http://www.jybaudot.fr/Analdonnees/cah.html>
- <http://www.jybaudot.fr/Stats/inertie.html>
- <https://www.rdocumentation.org/>
- <https://bupar.net/>
- Rapport de projet de recherche "Performance Analysis of the 2023 Women's World Cup using StatsBomb Open Source Data", Laly Lacroix et Julie Treilhou, INSA, aperçu disponible à cette adresse https://hal.science/hal-04574620v1/file/Lacroix_et_al_sport_process_mining.pdf

7 Annexe

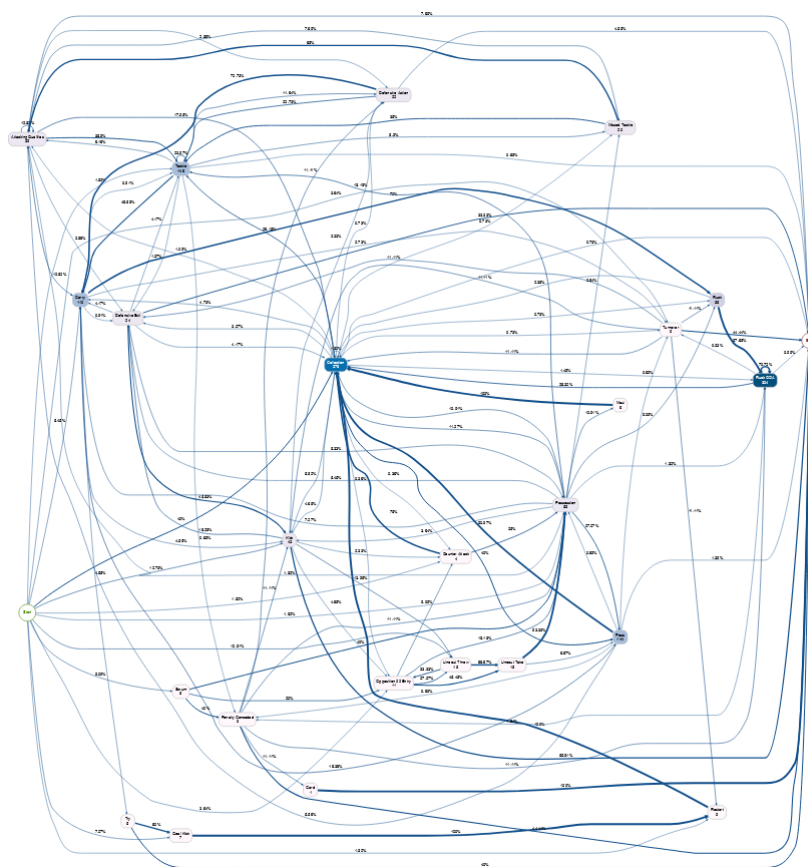


FIGURE 13 – Graphe sans clustering

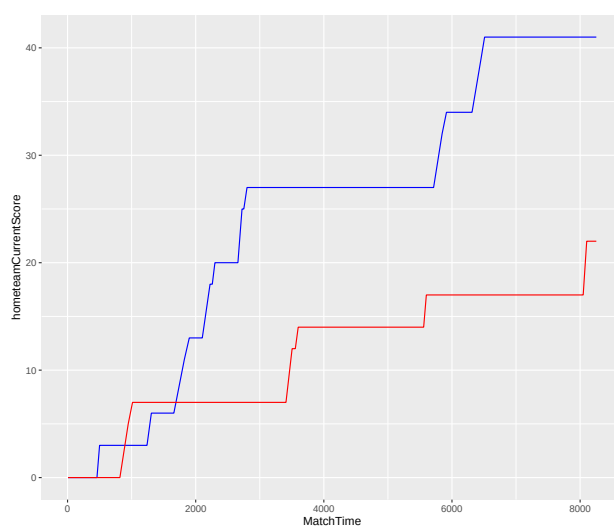


FIGURE 14 – Évolution du score en fonction du temps

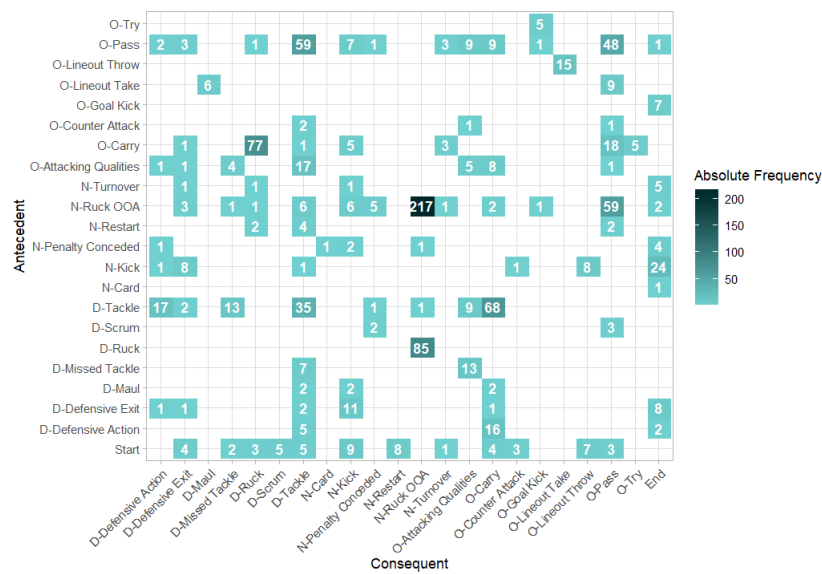


FIGURE 15 – Matrice de précedence de Leinster sans clustering

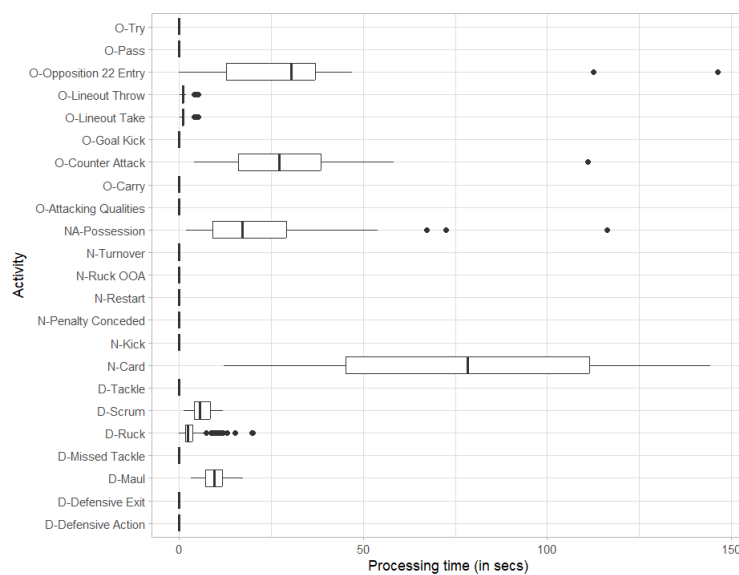


FIGURE 16 – Durée de chaque action

Two Sample t-test

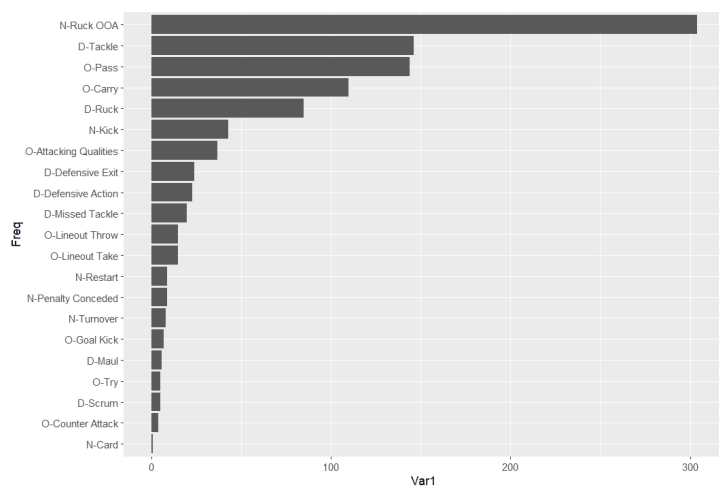
data: diff_distances_2300 and diff_distances_2350
 $t = 0.70223$, $df = 104$, $p\text{-value} = 0.4841$
 alternative hypothesis: true difference in means is not equal to 0
 95 percent confidence interval:
 -5.234357 10.974107
 sample estimates:
 mean of x mean of y
 26.90909 24.03922

FIGURE 17 – Test de Student non apparié de comparaison de moyennes entre les distances gagnées de Leinster et Stade Toulousain

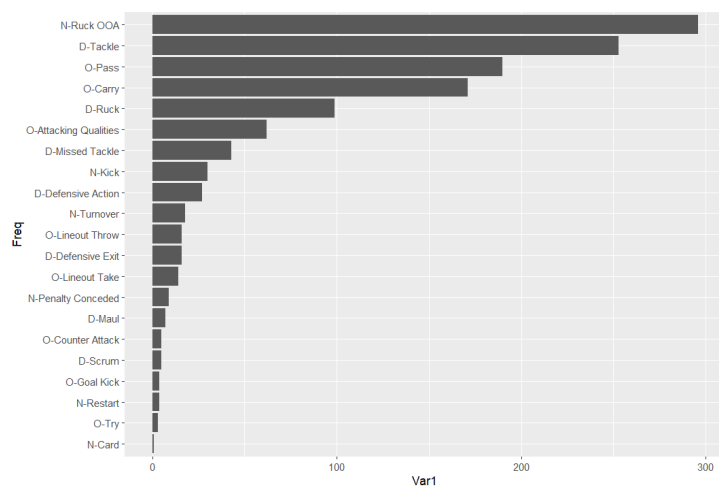
Welch Two Sample t-test

data: diff_distances_2300_r and diff_distances_2350_r
t = 4.7028, df = 14.221, p-value = 0.0001628
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
12.83073 Inf
sample estimates:
mean of x mean of y
20.5 0.0

FIGURE 18 – Test de Student-Welch non apparié de comparaison de moyennes entre les distances gagnées de Leinster et Stade Toulousain pendant la période critique



(a) Leinster



(b) Toulouse

FIGURE 19 – Nombre d'apparitions des actions